

Network Security

Handout: Docker & LXC

Introduction

Due to the nature of some of the assignments in this course, we use two container solutions called Docker and *Linux Containers* (LXC). In this handout, we introduce both solutions using step-by-step examples. On top of that, we also use Docker Compose. Docker Compose is a supplemental solution to automate the rollout of a container environment in Docker. We also explain the usage of this technology inside this handout.

Docker

Docker is a cross-platform container solution available on the following platforms:

- Windows
- Mac
- Linux

In the case of Windows and Mac, Docker can be downloaded and installed through the respective hyperlinks:

- **Windows:** <https://docs.docker.com/desktop/windows/install/>
- **Mac:** <https://docs.docker.com/desktop/mac/install/>

Docker Desktop contains both the Docker Engine as well as Docker Compose. In Linux, the following steps need to be taken to install both Docker components. We use Ubuntu 20.04 as an example.

```
sudo apt update && sudo apt-get install -y ca-certificates curl gnupg lsb-release

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o \
/usr/share/keyrings/docker-archive-keyring.gpg

echo \
"deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list \
> /dev/null

sudo apt update && sudo apt install -y docker-ce docker-ce-cli docker-compose \
containerd.io

# In order to access the Docker command without running sudo
sudo usermod -aG docker ${USER}
su - ${USER}
```

After installing Docker (and Docker Compose), execute the command below inside a terminal or command prompt (depending on the Operating System you are using).

```
docker run -ti ubuntu
```

This command creates a Docker container running Ubuntu. The `-ti` arguments are used to attach to the container a terminal session directly. We can exit the container by issuing the following command inside the container's terminal session.

```
exit
```

The command above also stops the container from running. To get an overview of the containers on a system, you can issue the following command.

```
docker ps -a
```

In our case, this gives us the following output.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a8499fc99561	ubuntu	"bash"	35 minutes ago	Exited (127)	31 minutes ago	dazzling_cohen

We issue the following command with the container's ID as an argument to start the container. One can find the container ID by executing the `docker ps -a` command.

```
docker start a8499fc99561
```

To connect to the container, we execute the following command. Again we use the container ID as one of the arguments.

```
docker exec -ti a8499fc99561 bash
```

If we exit the terminal session, the container won't stop running. This situation only happens when we stop the terminal session after issuing the `docker run -ti ubuntu` command.

To stop the container, we execute the following command. Like before, we use the container ID as an argument.

```
docker stop a8499fc99561
```

After the container is stopped, we can remove the container from the system using the command below.

```
docker rm a8499fc99561
```

With Docker Compose, we can script the creation of containers and networks. This solution allows us to generate complete environments based on a configuration (YAML) file. For example, copy the contents of the following code snippet to a file named `docker-compose.yml` inside an empty directory.

```

version: "3.9"

services:
  attacker:
    container_name: host01
    hostname: host01
    image: ubuntu
    command: tail -F anything
    networks:
      home_net:
        ipv4_address: 192.168.124.10
  defender:
    container_name: host02
    hostname: host02
    image: ubuntu
    command: tail -F anything
    networks:
      home_net:
        ipv4_address: 192.168.124.20

networks:
  home_net:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.124.0/24
          gateway: 192.168.124.1

```

Using a terminal or command prompt, navigate to the directory containing the `docker-compose.yml` file. Next, execute the command below to generate the environment inside the YAML file.

```

docker-compose build
docker-compose up -d

```

This command first builds the required containers and spins up the whole environment. These commands are two Ubuntu containers connected to the same bridge with a static IPv4 address.

We can enter one of the containers just like we did before when we created a container manually.

```

docker exec -ti 17ccfef0409 bash

```

To shutdown the environment, execute the command below.

```

docker-compose stop

```

Using the command below, we can also destroy the whole environment (including the bridge).

```

docker-compose down

```

LXC

Docker is sufficient for a lot of applications. However, this sometimes interferes with the internal Docker services when dealing with low-level network protocols. Therefore, we also use LXC for some purposes. From a functionality point of view, LXC is the same. However, one of the differences is that LXC is only supported on Linux Operating Systems. Therefore, if you don't run Linux, you can run a virtualized instance of Linux.

To run a virtualized instance of Linux, you can use Virtual Box. The following hyperlink explains how to install Virtual Box and Ubuntu.

<https://brb.nci.nih.gov/seqtools/installUbuntu.html>

Make sure to install Ubuntu 20.04 instead of Ubuntu 14 (as explained on the web page). Inside Ubuntu, open the terminal and execute the following commands to install LXC.

```
sudo apt update && sudo apt install -y lxc lxc-templates
```

After installing LXC, we can create our first container using the command below.

```
sudo lxc-create -t ubuntu -n host01
```

By default the container isn't started, we can create the container using the following command. `host01` is the name of the container we provided when creating the container earlier.

```
sudo lxc-start host01
```

We execute the command below to establish a terminal session inside the container.

```
sudo lxc-attach host01 bash
```

To stop the container, you need to execute the command below.

```
sudo lxc-stop host01
```

To altogether remove the container, we issue the following command.

```
sudo lxc-destroy host01
```